

Assignment - Optimization Techniques

Big-M Simplex Method & Transportation Problem (VAM + MODI)

1. Big-M Method

Problem: Minimize $Z = 4x_1 + x_2$, subject to $3x_1 + x_2 = 3$, $4x_1 + 3x_2 \geq 6$, $x_1 + 2x_2 \leq 4$, $x_1, x_2 \geq 0$.

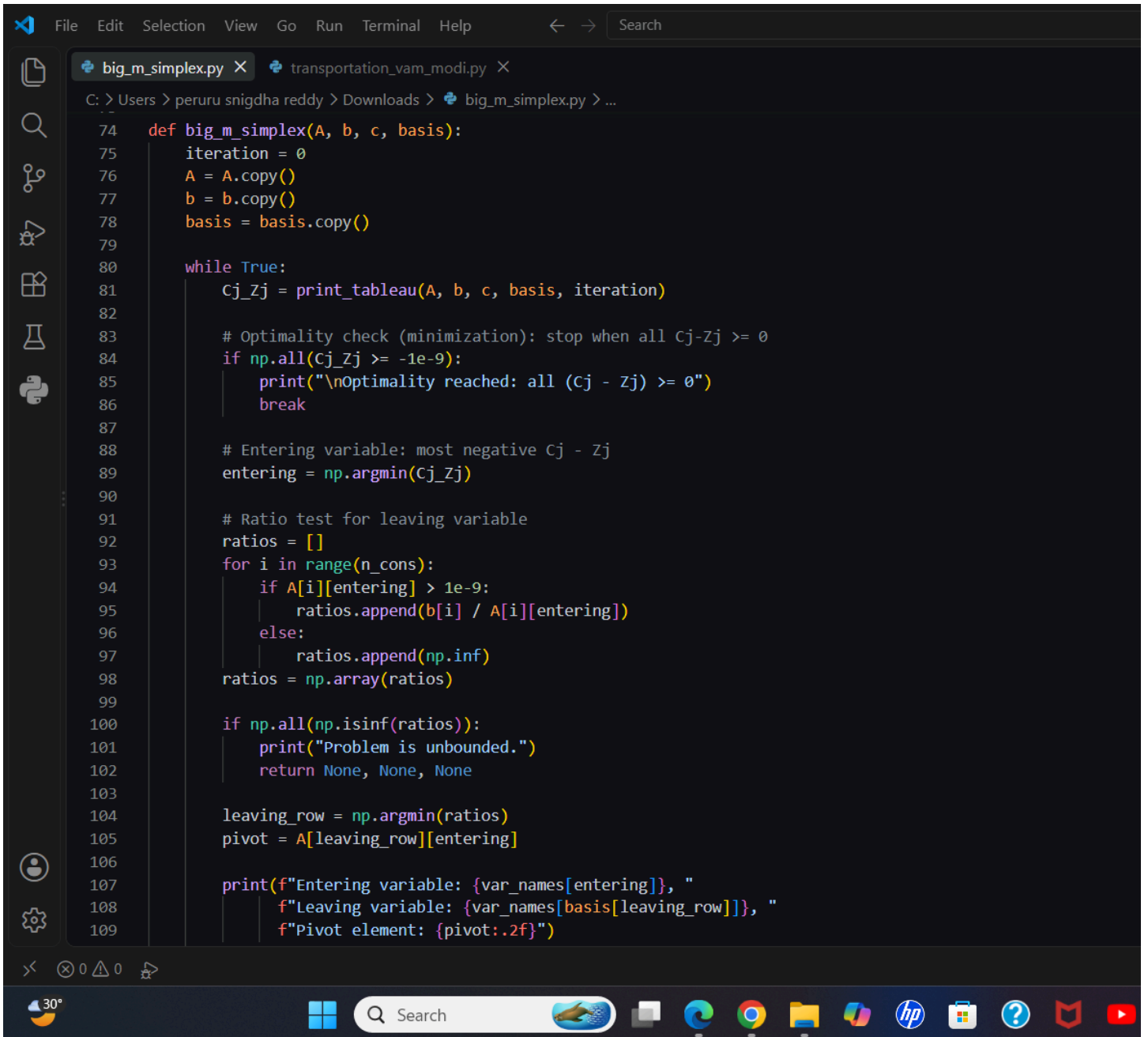
Formulated as an LPP in standard form using slack, surplus and artificial variables, and solved with the

Big-M Simplex Method

```
File Edit Selection View Go Run Terminal Help
big_m_simplex.py X transportation_vam_modi.py X
C: > Users > peruru snigdha reddy > Downloads > big_m_simplex.py > ...
1  """
2  Big-M Simplex Method
3  =====
4  Problem :
5
6      Minimize    Z = 4x1 + x2
7      Subject to:
8          3x1 + x2 = 3
9          4x1 + 3x2 >= 6
10         x1 + 2x2 <= 4
11         x1, x2 >= 0
12
13  Standard form conversion:
14      Constraint 1 (=) : needs an artificial variable A1
15      Constraint 2 (>=) : subtract surplus s1, add artificial variable A2
16      Constraint 3 (<=) : add slack s2
17
18      3x1 + x2 + A1 = 3
19      4x1 + 3x2 - s1 + A2 = 6
20      x1 + 2x2 + s2 = 4
21
22  Big-M objective (minimization):
23      Minimize Z = 4x1 + x2 + M*A1 + M*A2
24      (M is a very large positive number)
25
26  This program builds the initial simplex tableau, then performs the
27  Big-M simplex iterations until optimality, printing every tableau.
28  """
29
30  import numpy as np
31
32  np.set_printoptions(precision=3, suppress=True)
33
34  M = 1000 # a sufficiently large number to represent Big-M
35
36  # Variable order: x1, x2, s1(surplus), A1, A2, s2(slack)
37  var_names = ["x1", "x2", "s1", "A1", "A2", "s2"]
```

```
File Edit Selection View Go Run Terminal Help
big_m_simplex.py X transportation_vam_modi.py X

C: > Users > peruru snigdha reddy > Downloads > big_m_simplex.py > ...
35
36 # Variable order: x1, x2, s1(surplus), A1, A2, s2(slack)
37 var_names = ["x1", "x2", "s1", "A1", "A2", "s2"]
38
39 # Objective coefficients (minimization): x1, x2, s1, A1, A2, s2
40 c = np.array([4, 1, 0, M, M, 0], dtype=float)
41
42 # Constraint matrix (rows = constraints)
43 A = np.array([
44     [3, 1, 0, 1, 0, 0], # 3x1 + x2 + A1 = 3
45     [4, 3, -1, 0, 1, 0], # 4x1 + 3x2 - s1 + A2 = 6
46     [1, 2, 0, 0, 0, 1], # x1 + 2x2 + s2 = 4
47 ], dtype=float)
48
49 b = np.array([3, 6, 4], dtype=float)
50
51 # Basic variables initially: A1, A2, s2 -> columns index 3, 4, 5
52 basis = [3, 4, 5]
53
54 n_vars = len(var_names)
55 n_cons = A.shape[0]
56
57
58 def print_tableau(A, b, c, basis, iteration):
59     print(f"\n--- Iteration {iteration} ---")
60     header = "Basis\t" + "\t".join(var_names) + "\tRHS"
61     print(header)
62     for i in range(n_cons):
63         row = f"{var_names[basis[i]]}\t" + "\t".join(f"{v:6.2f}" for v in A[i]) + f"\t{b[i]:6.2f}"
64         print(row)
65     # Zj row and Cj - Zj row
66     Zj = np.array([sum(c[basis[i]] * A[i][j]) for i in range(n_cons)] for j in range(n_vars)]
67     Cj_Zj = c - Zj
68     Zval = sum(c[basis[i]] * b[i] for i in range(n_cons))
69     print("Zj\t" + "\t".join(f"{v:6.2f}" for v in Zj) + f"\t{Zval:6.2f}")
70     print("Cj-Zj\t" + "\t".join(f"{v:6.2f}" for v in Cj_Zj))
71     return Cj_Zj
```



```
File Edit Selection View Go Run Terminal Help
big_m_simplex.py X transportation_vam_modi.py X
C: > Users > peruru snigdha reddy > Downloads > big_m_simplex.py > ...
74 def big_m_simplex(A, b, c, basis):
111     # Pivot operation
112     A[leaving_row] = A[leaving_row] / pivot
113     b[leaving_row] = b[leaving_row] / pivot
114     for i in range(n_cons):
115         if i != leaving_row:
116             factor = A[i][entering]
117             A[i] = A[i] - factor * A[leaving_row]
118             b[i] = b[i] - factor * b[leaving_row]
119
120     basis[leaving_row] = entering
121     iteration += 1
122
123     if iteration > 20:
124         print("Stopping: too many iterations.")
125         break
126
127     return A, b, basis
128
129
130 if __name__ == "__main__":
131     A_final, b_final, basis_final = big_m_simplex(A, b, c, basis)
132
133     if A_final is not None:
134         print("\n===== OPTIMAL SOLUTION =====")
135         solution = {name: 0.0 for name in var_names}
136         for i, bi in enumerate(basis_final):
137             solution[var_names[bi]] = b_final[i]
138
139         for name in var_names:
140             print(f"{name} = {solution[name]:.3f}")
141
142         # Check no artificial variable remains positive in basis (feasibility)
143         if solution["A1"] > 1e-6 or solution["A2"] > 1e-6:
144             print("\nWarning: Artificial variable is positive -> No feasible solution exists.")
145         else:
```

```
big_m_simplex.py X transportation_vam_modi.py X
C: > Users > peruru snigdha reddy > Downloads > big_m_simplex.py > ...
145     else:
146         Z_optimal = 4 * solution["x1"] + 1 * solution["x2"]
147         print(f"\nOptimal Objective Value: Z = 4*x1 + x2 = {Z_optimal:.3f}")
148
```

OUTPUT:

```

File Edit Selection View Go Run Terminal Help
Problems Output Debug Console Terminal Ports

--- Iteration 0 ---
Basis  x1  x2  s1  A1  A2  s2  RHS
A1      3.00  1.00  0.00  1.00  0.00  0.00  3.00
A2      4.00  3.00 -1.00  0.00  1.00  0.00  6.00
s2      1.00  2.00  0.00  0.00  0.00  1.00  4.00
Zj      7000.00 4000.00 -1000.00 1000.00 1000.00 0.00 9000.00
Cj-Zj   -6996.00 -3999.00 1000.00 0.00 0.00 0.00 0.00
Entering variable: x1, Leaving variable: A1, Pivot element: 3.00

--- Iteration 1 ---
Basis  x1  x2  s1  A1  A2  s2  RHS
x1      1.00  0.33  0.00  0.33  0.00  0.00  1.00
A2      0.00  1.67 -1.00 -1.33  1.00  0.00  2.00
s2      0.00  1.67  0.00 -0.33  0.00  1.00  3.00
Zj      4.00 1668.00 -1000.00 -1332.00 1000.00 0.00 2004.00
Cj-Zj   0.00 -1667.00 1000.00 2332.00 0.00 0.00 0.00
Entering variable: x2, Leaving variable: A2, Pivot element: 1.67

--- Iteration 2 ---
Basis  x1  x2  s1  A1  A2  s2  RHS
x1      1.00  0.00  0.20  0.60 -0.20  0.00  0.60
x2      0.00  1.00 -0.60 -0.80  0.60  0.00  1.20
s2      0.00  0.00  1.00  1.00 -1.00  1.00  1.00
Zj      4.00  1.00  0.20  1.60 -0.20  0.00  3.60
Cj-Zj   0.00  0.00 -0.20 998.40 1000.20 0.00 0.00
Entering variable: s1, Leaving variable: s2, Pivot element: 1.00

--- Iteration 3 ---
Basis  x1  x2  s1  A1  A2  s2  RHS
x1      1.00  0.00  0.00  0.40  0.00 -0.20  0.40
x2      0.00  1.00  0.00 -0.20  0.00  0.60  1.80
s1      0.00  0.00  1.00  1.00 -1.00  1.00  1.00
Zj      4.00  1.00  0.00  1.40  0.00 -0.20  3.40
Cj-Zj   0.00  0.00  0.00 998.60 1000.00 0.20 0.00

Optimality reached: all (Cj - Zj) >= 0

===== OPTIMAL SOLUTION =====

```

```

===== OPTIMAL SOLUTION =====
x1 = 0.400
x2 = 1.800
s1 = 1.000
A1 = 0.000
A2 = 0.000
s2 = 0.000

Optimal Objective Value: Z = 4*x1 + x2 = 3.400
PS C:\Users\peruru snigdha reddy\Downloads>

```

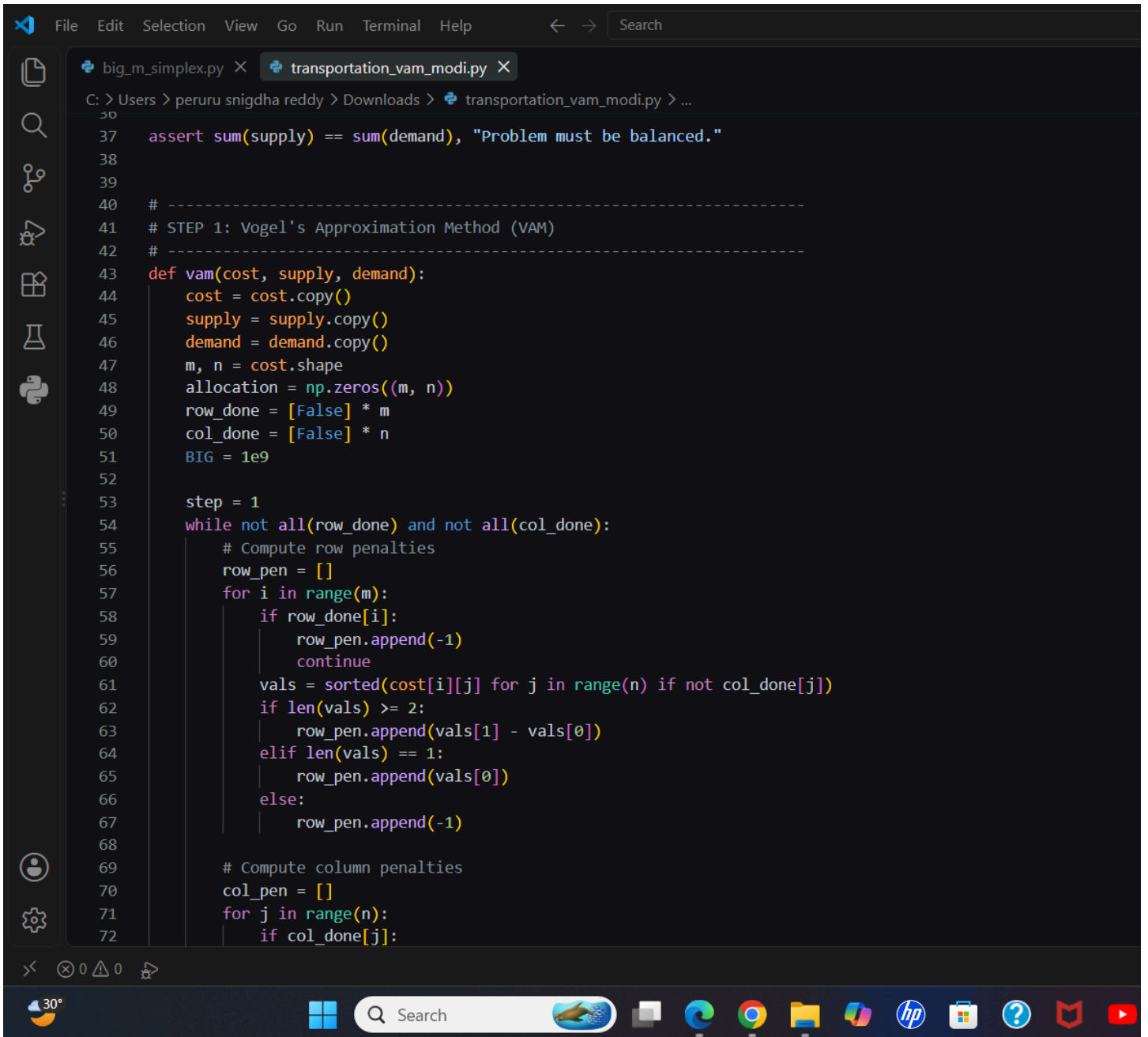
2. Transportation Problem – VAM and MODI

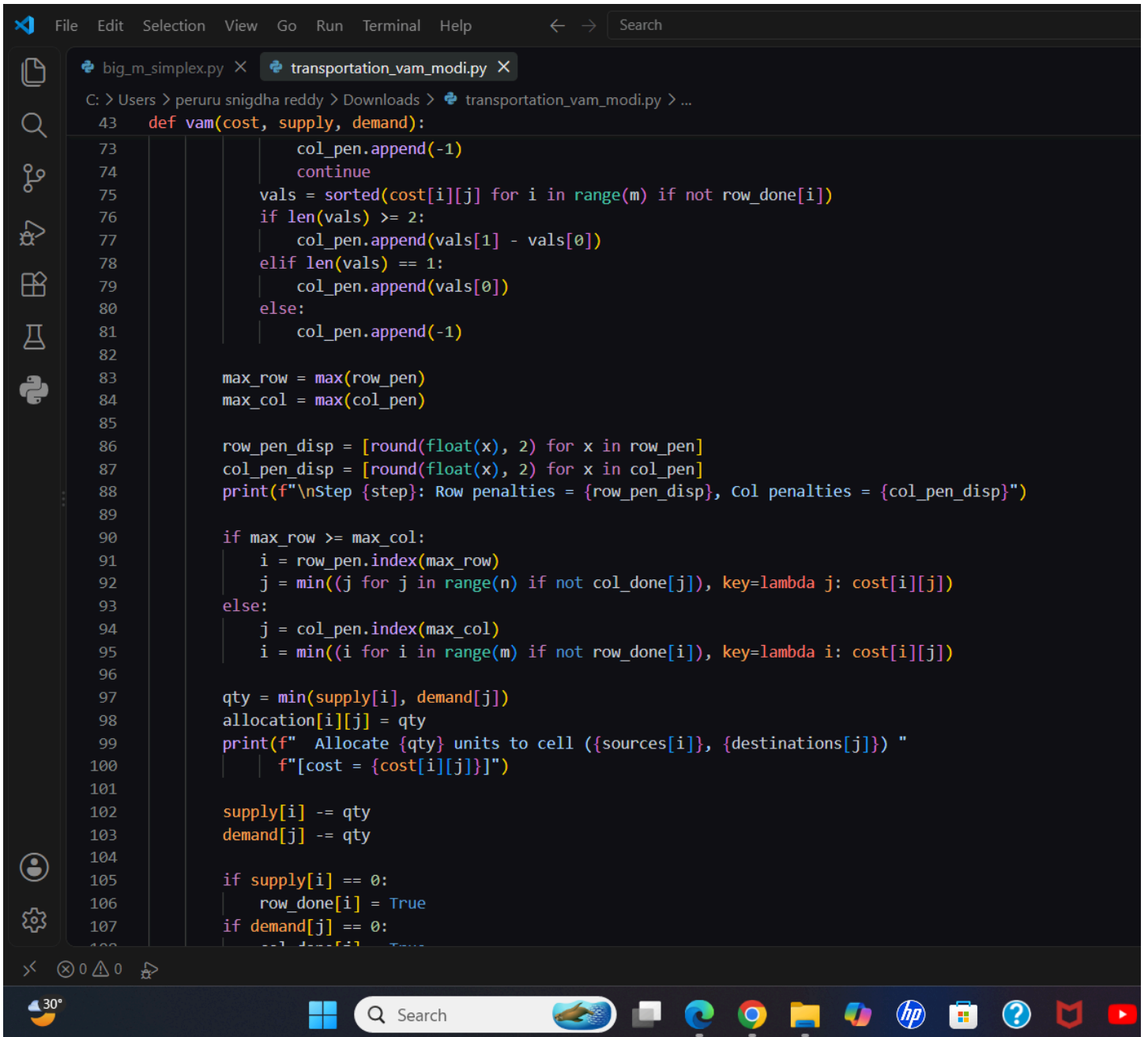
Case study: a balanced transportation problem with 3 sources (supply 50, 60, 25) and 4 destinations

(demand 30, 40, 50, 15). Vogel's Approximation Method (VAM) is used to obtain an initial basic feasible

solution, and the MODI (u-v) method is then used to test optimality and improve the allocation

```
File Edit Selection View Go Run Terminal Help
big_m_simplex.py X transportation_vam_modi.py X
C: > Users > peruru snigdha reddy > Downloads > transportation_vam_modi.py > ...
1  """
2  Transportation Problem: Vogel's Approximation Method (VAM) + MODI Method
3  =====
4
5  Case study :
6
7  |   |   |   |   |   |   |
8  |   |   | D1 | D2 | D3 | D4 | Supply
9  |   |   |   |   |   |   |
10 |   |   |   |   |   |   |
11 |   |   |   |   |   |   |
12 Demand      30  40  50  15  |  135 (= total supply)
13
14 Step 1: Use VAM to obtain an initial basic feasible solution (IBFS).
15 Step 2: Use MODI (u-v method) to test the IBFS for optimality and,
16         if needed, iteratively improve the allocation until the
17         optimal (minimum-cost) solution is reached.
18 """
19
20 import numpy as np
21 import copy
22
23 np.set_printoptions(suppress=True)
24
25 sources = ["01", "02", "03"]
26 destinations = ["D1", "D2", "D3", "D4"]
27
28 cost = np.array([
29     [4, 6, 8, 6],
30     [3, 5, 2, 5],
31     [3, 9, 6, 5],
32 ], dtype=float)
33
34 supply = [50, 60, 25]
35 demand = [30, 40, 50, 15]
36
37 assert sum(supply) == sum(demand), "Problem must be balanced."
```



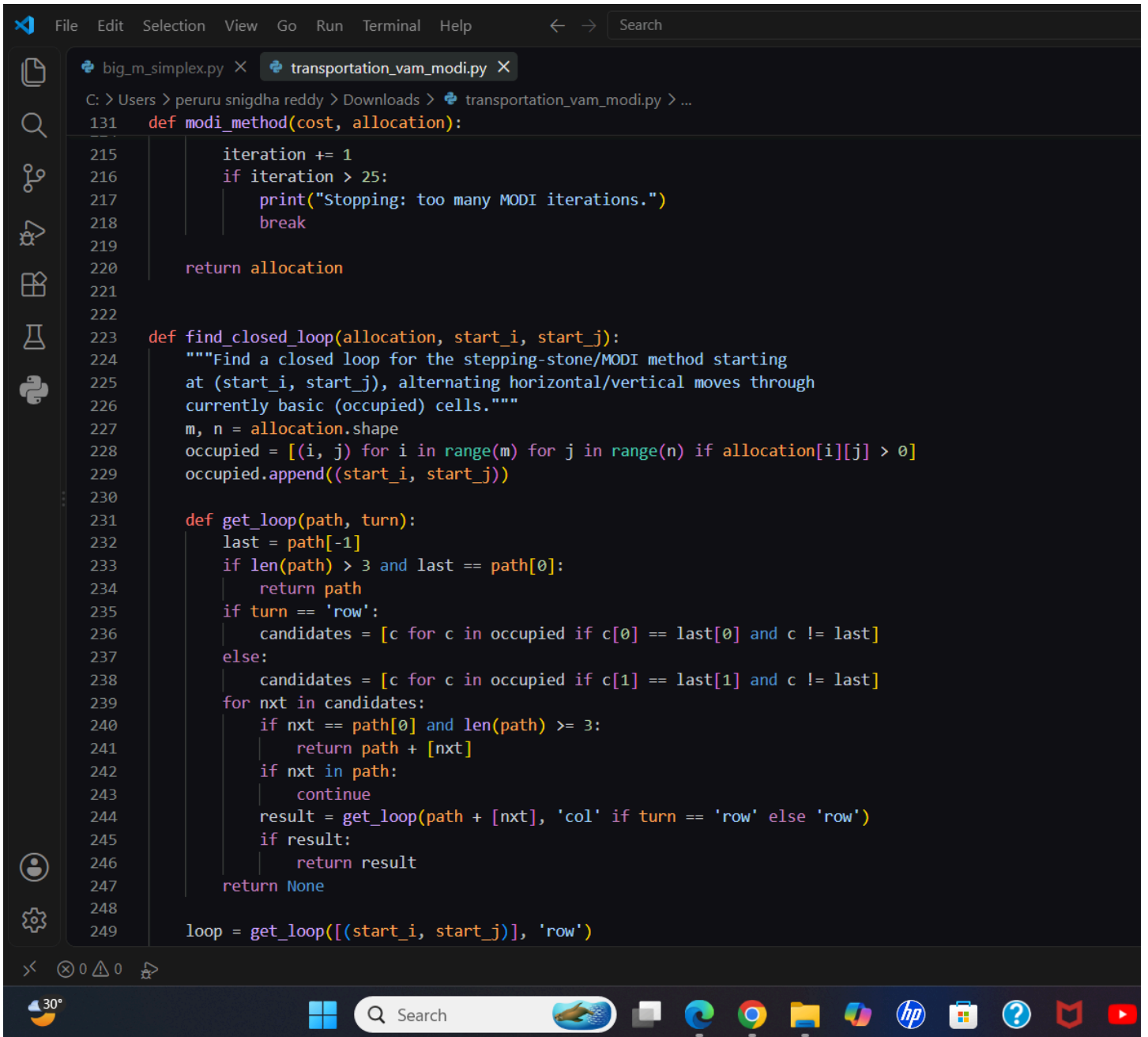


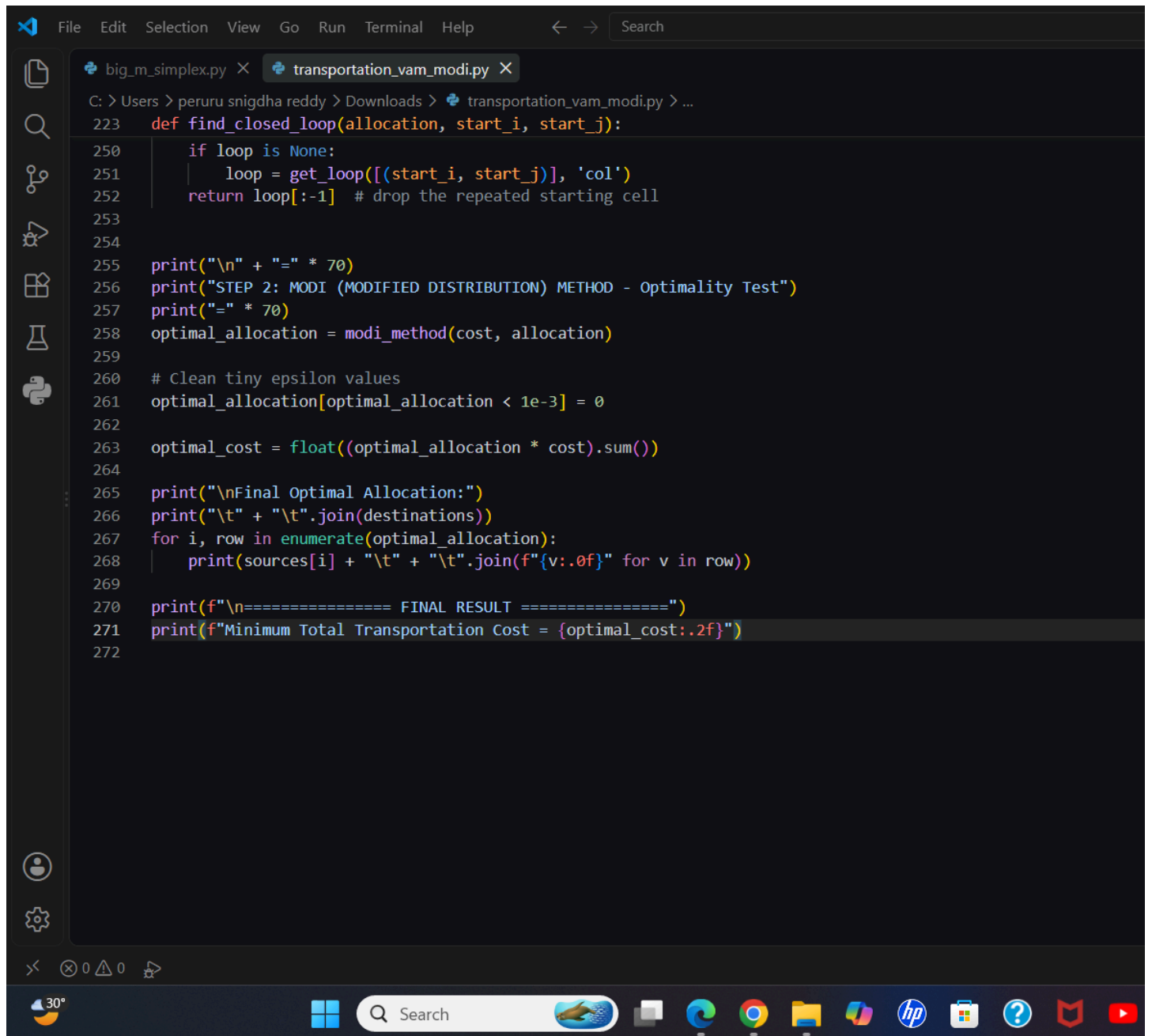
```
File Edit Selection View Go Run Terminal Help
big_m_simplex.py X transportation_vam_modi.py X
C: > Users > peruru snigdha reddy > Downloads > transportation_vam_modi.py > ...
43 def vam(cost, supply, demand):
108     col_done[j] = True
109
110     step += 1
111
112     total_cost = float((allocation * (cost)).sum())
113     return allocation, total_cost
114
115
116 print("=" * 70)
117 print("STEP 1: VOGEL'S APPROXIMATION METHOD (VAM) - Initial Basic Feasible Solution")
118 print("=" * 70)
119 allocation, ibfs_cost = vam(cost, supply, demand)
120
121 print("\nInitial Allocation (VAM):")
122 print("\t" + "\t".join(destinations))
123 for i, row in enumerate(allocation):
124     print(sources[i] + "\t" + "\t".join(f"{v:.0f}" for v in row))
125 print(f"\nTotal Transportation Cost (IBFS from VAM) = {ibfs_cost:.2f}")
126
127
128 # -----
129 # STEP 2: MODI (Modified Distribution) Method - test/optimize
130 # -----
131 def modi_method(cost, allocation):
132     m, n = cost.shape
133     allocation = allocation.copy()
134     iteration = 1
135
136     while True:
137         print(f"\n--- MODI Iteration {iteration} ---")
138         basic_cells = [(i, j) for i in range(m) for j in range(n) if allocation[i][j] > 0]
139
140         # Handle degeneracy: need m+n-1 basic cells
141         needed = m + n - 1
142         if len(basic_cells) < needed:
143             # add an empty allocation to the lowest cost independent empty cell
```

```
File Edit Selection View Go Run Terminal Help
big_m_simplex.py X transportation_vam_modi.py X

C: > Users > peruru snigdha reddy > Downloads > transportation_vam_modi.py > ...
131 def modi_method(cost, allocation):
143     # add an epsilon allocation to the lowest-cost independent empty cell
144     for i in range(m):
145         for j in range(n):
146             if allocation[i][j] == 0 and (i, j) not in basic_cells:
147                 allocation[i][j] = 1e-6
148                 basic_cells.append((i, j))
149                 print(f" Degeneracy handled: added epsilon at ({sources[i]}, {destinations[j]})")
150                 break
151             if len(basic_cells) >= needed:
152                 break
153
154     # Solve for u_i, v_j using u_i + v_j = c_ij for basic cells
155     u = [None] * m
156     v = [None] * n
157     u[0] = 0
158     changed = True
159     while changed:
160         changed = False
161         for (i, j) in basic_cells:
162             if u[i] is not None and v[j] is None:
163                 v[j] = cost[i][j] - u[i]
164                 changed = True
165             elif v[j] is not None and u[i] is None:
166                 u[i] = cost[i][j] - v[j]
167                 changed = True
168
169     print(f" u = {[ '%.2f'%x if x is not None else None for x in u ]}")
170     print(f" v = {[ '%.2f'%x if x is not None else None for x in v ]}")
171
172     # Compute opportunity cost (Cij - ui - vj) for non-basic cells
173     delta = np.full((m, n), np.nan)
174     for i in range(m):
175         for j in range(n):
176             if (i, j) not in basic_cells:
177                 delta[i][j] = cost[i][j] - u[i] - v[j]
178
```

```
File Edit Selection View Go Run Terminal Help
big_m_simplex.py X transportation_vam_modi.py X
C: > Users > peruru snigdha reddy > Downloads > transportation_vam_modi.py > ...
131 def modi_method(cost, allocation):
179     print(" Opportunity costs (delta) for non-basic cells:")
180     for i in range(m):
181         row_str = []
182         for j in range(n):
183             if np.isnan(delta[i][j]):
184                 row_str.append(" - ")
185             else:
186                 row_str.append(f"{delta[i][j]:5.2f}")
187         print(" " + "\t".join(row_str))
188
189     # Optimality check: all deltas >= 0
190     min_delta = np.nanmin(delta)
191     if min_delta >= -1e-6 or np.isnan(min_delta):
192         print("\n All opportunity costs >= 0 -> OPTIMAL SOLUTION REACHED.")
193         break
194
195     # Entering cell = most negative delta
196     enter_i, enter_j = np.unravel_index(np.nanargmin(delta), delta.shape)
197     print(f" Entering cell: ({sources[enter_i]}, {destinations[enter_j]}) "
198           f"with delta = {delta[enter_i][enter_j]:.2f}")
199
200     # Build closed loop (stepping-stone) starting at entering cell
201     loop = find_closed_loop(allocation, enter_i, enter_j)
202     print(f" Closed loop: {[sources[i], destinations[j]] for i, j in loop}")
203
204     # Alternate +/- around loop; theta = min allocation at '-' positions
205     minus_cells = loop[1::2]
206     theta = min(allocation[i][j] for i, j in minus_cells)
207     print(f" Theta (units to reallocate) = {theta:.2f}")
208
209     for idx, (i, j) in enumerate(loop):
210         if idx % 2 == 0:
211             allocation[i][j] += theta
212         else:
213             allocation[i][j] -= theta
```

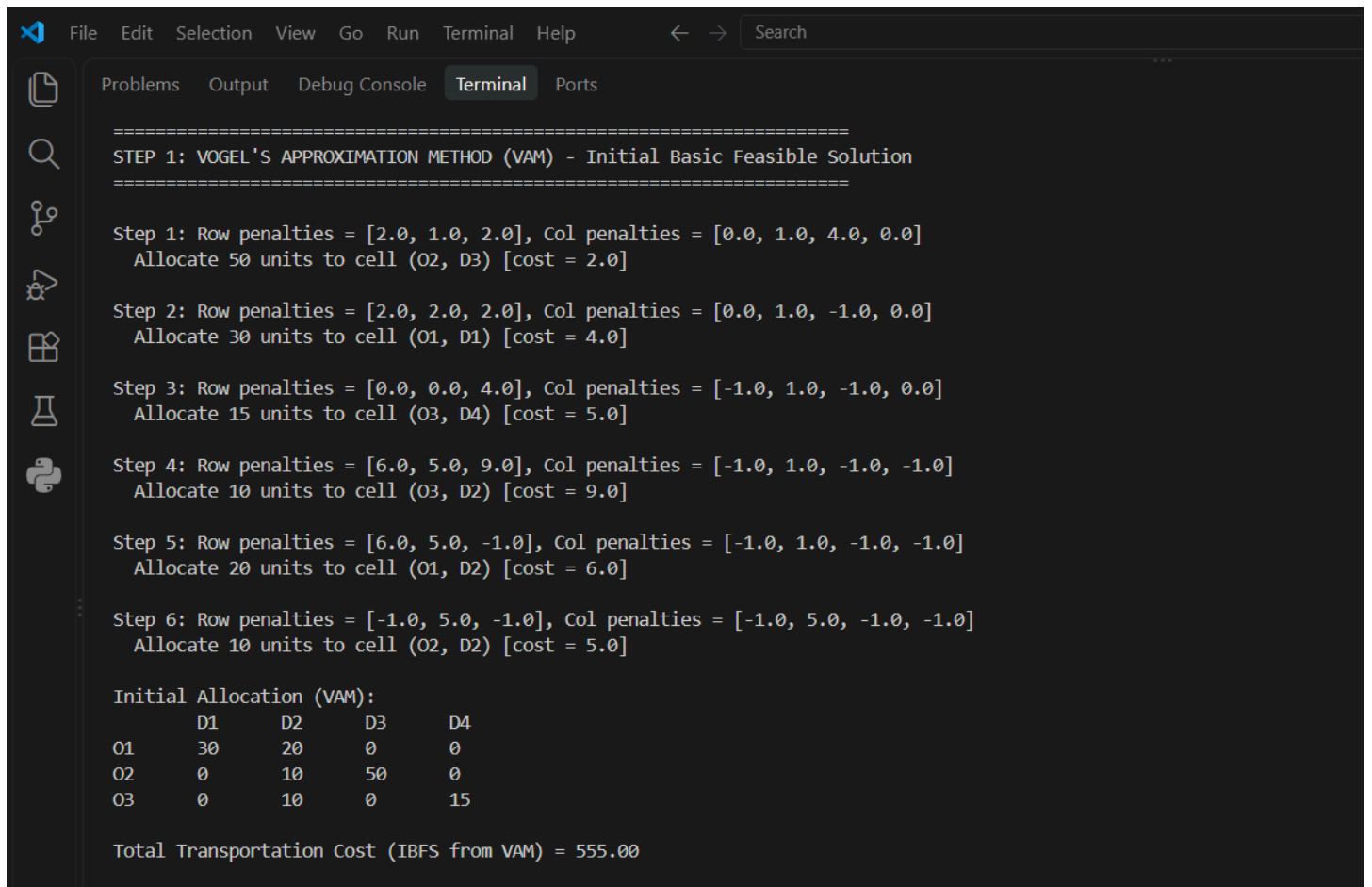




```
File Edit Selection View Go Run Terminal Help
big_m_simplex.py X transportation_vam_modi.py X

C: > Users > peruru snigdha reddy > Downloads > transportation_vam_modi.py > ...
223 def find_closed_loop(allocation, start_i, start_j):
250     if loop is None:
251         loop = get_loop([(start_i, start_j)], 'col')
252         return loop[:-1] # drop the repeated starting cell
253
254
255 print("\n" + "=" * 70)
256 print("STEP 2: MODI (MODIFIED DISTRIBUTION) METHOD - Optimality Test")
257 print("=" * 70)
258 optimal_allocation = modi_method(cost, allocation)
259
260 # Clean tiny epsilon values
261 optimal_allocation[optimal_allocation < 1e-3] = 0
262
263 optimal_cost = float((optimal_allocation * cost).sum())
264
265 print("\nFinal Optimal Allocation:")
266 print("\t" + "\t".join(destinations))
267 for i, row in enumerate(optimal_allocation):
268     print(sources[i] + "\t" + "\t".join(f"{v:.0f}" for v in row))
269
270 print(f"\n===== FINAL RESULT =====")
271 print(f"Minimum Total Transportation Cost = {optimal_cost:.2f}")
272
```

OUTPUT:



File Edit Selection View Go Run Terminal Help

Problems Output Debug Console **Terminal** Ports

```
=====
STEP 2: MODI (MODIFIED DISTRIBUTION) METHOD - Optimality Test
=====

--- MODI Iteration 1 ---
u = ['0.00', '-1.00', '3.00']
v = ['4.00', '6.00', '3.00', '2.00']
Opportunity costs (delta) for non-basic cells:
-      -      5.00      4.00
0.00   -      -      4.00
-4.00  -      0.00   -
Entering cell: (03, D1) with delta = -4.00
Closed loop: [('03', 'D1'), ('03', 'D2'), ('01', 'D2'), ('01', 'D1')]
Theta (units to reallocate) = 10.00

--- MODI Iteration 2 ---
u = ['0.00', '-1.00', '-1.00']
v = ['4.00', '6.00', '3.00', '6.00']
Opportunity costs (delta) for non-basic cells:
-      -      5.00      0.00
0.00   -      -      0.00
-      4.00      4.00   -

All opportunity costs >= 0 -> OPTIMAL SOLUTION REACHED.

Final Optimal Allocation:
      D1      D2      D3      D4
01      20      30      0       0
02      0       10      50      0
03      10      0       0      15

===== FINAL RESULT =====
Minimum Total Transportation Cost = 515.00
PS C:\Users\peruru snigdha reddy\Downloads> 
```

30°

Search

hp

?

YouTube